DEEP LEARNING-BASED VOICE EMOTION RECOGNITION

Complete All-Features Project Blueprint

Core model: CNN–LSTM with audio features
Full target feature set: emotion timeline, uncertainty-aware predictions, robustness testing, explainability, speaker adaptation, response suggestions, dashboard, advanced-model comparisons and deployment pathway

Academic project design • 2026–27
RV University • School of Computer Science and Engineering • Bengaluru

This edition treats every feature discussed in the project design as part of the full target system. A staged build order is included because some features depend on earlier components and may require additional time or compute. Proposed features are not represented as already implemented or proven.

1. Full feature set — scope of the project

Project title remains **Deep Learning-Based Voice Emotion Recognition**. The enhanced product concept can be called **EmotionLens** for the dashboard/demo, while the official academic title remains unchanged.

ID	Feature	What the finished system should do	Visible deliverable
F01	Audio upload and validation	Accept supported WAV files; check sample rate, channels, duration, silence and corrupt files	Upload panel + useful errors
F02	Audio preprocessing	Mono conversion, resampling, consistent normalisation, silence handling and optional denoising	Preprocessed audio + settings
F03	Feature extraction	Generate log-Mel spectrograms and MFCC; calculate selected pitch, energy, speaking-rate and spectral descriptors	Spectrogram and feature summary
F04	CNN–LSTM classifier	Learn local time-frequency patterns with CNN and temporal dependencies with LSTM	Predicted class scores
F05	Emotion timeline	Split audio into timestamped windows and predict each segment	Timeline with segment labels and scores
F06	Uncertainty handling	Use validation-selected threshold to flag ambiguous/low-confidence predictions	Uncertain state instead of forced label
F07	Noise robustness	Test clean/noisy audio and optional denoising; chart performance under controlled noise	Robustness plot + comparison
F08	Speaker-independent evaluation	Keep speakers disjoint between training, validation and test where possible	Split audit + held-out metrics
F09	Explainability	Display waveform, spectrogram, acoustic summaries and optional attribution heatmap	Evidence/visualisation panel
F10	Speaker-adaptive baseline	With consent, compare a speaker's current features with a reference baseline	Personal-relative change view
F11	Emotion-aware response suggestions	Offer a reviewed, rule/template-based communication suggestion from the model output	Suggested response with caveat
F12	Interactive dashboard	Connect upload, playback, results, timeline, uncertainty, visualisations and evaluation page	Working app
F13	Model comparison	Compare CNN, CNN–LSTM, and if feasible BiLSTM/attention or pretrained audio embeddings	Fair comparison table
F14	Evaluation and reporting	Compute accuracy, macro F1, class-wise metrics, confusion matrix, calibration/coverage and latency	Reproducible report
F15	Data and model versioning	Save configuration, split IDs, class map, library versions, checkpoints and model version	Experiment log + model card
F16	Testing and error handling	Test invalid input, shapes, timestamps, inference, uncertain cases and UI flow	Test checklist/log
F17	Deployment pathway	Package inference behind a local web app/API; document requirements and deployment steps	Runnable demo/package
F18	Privacy and responsible use	Consent, local-first processing, minimal retention, caveats and restricted use guidance	Privacy notice + limitations
F19	Advanced extension: m ultilingual/cross-corpus	Test a defensible shared label set across datasets/languages where data permit	Cross-corpus results
F20	Advanced extension: multimodal fusion	Optional voice + text or video comparison only with suitable consented data and a separate evaluation design	Fusion experiment

All F01–F18 are part of the full target design. F19–F20 are also documented as advanced extensions; include them in the final implementation only if the team has suitable data, time and compute.

2. End-to-end architecture

The system is organised as independent modules so that the model, timeline logic, uncertainty layer and user interface can be tested separately.

Stage	Operation	Output
1. Input	WAV upload or consented microphone recording	Audio + metadata
2. Validate	File type, duration, sample rate, channels, empty/corrupt file checks	Accepted audio or error
3. Standardise	Mono conversion, target sample rate, normalisation and optional denoising	Consistent waveform
4. Segment	Voice activity/silence handling and overlapping fixed-duration windows	Segments + timestamps
5. Features	Log-Mel spectrogram/MFCC plus optional prosodic descriptors	Model-ready tensors + features
6. Predict	CNN feature extractor → LSTM temporal model → dense/softmax head	Per-segment class scores
7. Reliability	Validation-based threshold; optional calibration/OOD checks if implemented	Predicted class or uncertain flag
8. Explain	Waveform, spectrogram, acoustic summaries and optional saliency map	Interpretability aids
9. Personalise	Compare against a consented speaker reference, if enabled	Relative acoustic changes
10. Respond	Map sufficiently supported output to a neutral response template	Optional reviewed suggestion
11. Present	Timeline, summary, plots, evaluation and export	Dashboard/report

Important: predictions are estimates of dataset-defined vocal emotion labels. They are not diagnoses and do not prove what a speaker truly feels.

Core CNN–LSTM design

Layer	Design choice	Purpose
Input	Log-Mel spectrogram (time × frequency)	Represent audio energy across time and frequency
CNN blocks	Convolution → activation → pooling; optional batch normalisation/dropout	Learn local acoustic patterns
Sequence reshape	Convert pooled features into a sequence	Preserve temporal ordering for recurrent model
LSTM	LSTM baseline; BiLSTM only as a comparison	Model dependencies across frames
Dense head	Dense + dropout	Combine learned representation
Output	Softmax over mapped target classes	Class scores for each segment

Model alternatives to evaluate

- Baseline 1: CNN-only model, to test whether recurrent temporal modelling helps.
- Baseline 2: CNN–LSTM, the primary project model.
- Advanced comparison: CNN–BiLSTM with attention, if feasible.
- Advanced comparison: embeddings from a suitable pretrained speech/audio model, subject to licence and compute constraints.
- Feature ablation: Mel-spectrogram alone versus Mel-spectrogram plus prosodic features.

3. Detailed design for the differentiating features

F05 — Emotion timeline

- Divide the recording into fixed-duration windows with a documented overlap.
- Retain each window's start/end time and audio segment reference.
- Run each segment through the same preprocessing and trained classifier.
- Plot predicted class per segment and model score over time; allow segment replay.
- Show overall summary separately from the timeline so the user can see local changes.
- Optionally smooth rapid class changes, but retain raw predictions for audit and explain the smoothing rule.
- State that timestamp-level emotion is a model estimate unless frame/segment-level ground truth exists.

F06 — Uncertainty-aware output

- Start with maximum predicted class score as a simple uncertainty signal.
- Choose the reject threshold using validation data only; document the criterion.
- If below threshold, show “Uncertain prediction” plus the leading class candidates.
- Report coverage versus error/selective risk; test calibration if time permits.
- Handle no-speech and out-of-distribution audio separately where the system supports these checks.
- Do not label raw softmax values as calibrated probabilities unless calibration has been evaluated.

F07/F08 — Robustness and generalisation

Test	Method	What to report
Clean baseline	Evaluate held-out speakers on clean test audio	Macro F1, per-class recall, confusion matrix
Noise stress test	Add controlled background noise at documented levels	Performance by noise level
Denoising ablation	Compare noisy input with and without denoising	Whether denoising improves or damages predictions
Speaker-independent split	Keep speaker IDs disjoint between splits	Performance on unseen speakers
Cross-corpus (advanced)	Train/test across corpora with defensible shared labels	Generalisation gap + label caveats

F09 — Explainability

- Show waveform and Mel-spectrogram for each uploaded recording.
- Display pitch, energy, speaking-rate proxy and selected spectral summaries.
- Optional: add an attribution/activation heatmap over time-frequency regions if the method is supported by the model.
- Describe these as signals associated with the prediction, not proof that a person is experiencing a particular emotion.

F10 — Speaker adaptation

- Ask for a separate reference recording only with informed consent.
- Calculate baseline acoustic summaries using the same processing settings.
- Compare current features with the reference and show relative changes, not a new certainty claim.
- Evaluate whether the baseline improves results across speakers; provide a way to disable/delete the reference.
- Treat as an advanced module because it raises data sufficiency, privacy and bias concerns.

4. Response suggestions, dashboard and deployment

F11 — Emotion-aware response suggestions

The application may offer a neutral communication suggestion based on a sufficiently supported model output. Use a predefined template library for the first version; do not allow the system to make high-stakes decisions or assert a person's internal state.

Model output	Example suggestion style
Potential frustration	“Consider acknowledging the concern and asking what help would be most useful.”
Potential sadness	“Consider responding gently and giving the speaker room to explain.”
Potential excitement / happiness	“Consider matching the positive tone while staying attentive to the actual words.”
Uncertain / low confidence	“No reliable suggestion is available from this audio. Consider asking a neutral clarifying question.”

F12 — Dashboard screens

Screen / panel	Components
Home / input	Project title, WAV upload, optional microphone capture, duration/sample-rate metadata, playback and input validation
Audio analysis	Waveform, Mel-spectrogram, optional MFCC/prosodic feature views
Prediction summary	Dominant predicted class, class-score distribution, uncertainty status and clear caveat
Timeline	Timestamped segment cards, emotion track, confidence trend and click-to-replay
Reliability	Noise warning if supported, uncertain-segment count, limitations and model version
Explainability	Acoustic feature summaries and optional model attribution
Speaker reference	Consent prompt, baseline upload, relative comparison and delete/reset option
Response suggestion	Template suggestion with review/edit controls; disabled when uncertain
Evaluation	Confusion matrix, per-class precision/recall, macro F1, training curves, noise plots and comparison results
Export	Summary report with timestamped predictions, settings and caveat; do not export raw audio by default

F17 — Deployment pathway

- First target: a local dashboard that runs on the project laptop and uses a saved model checkpoint.
- Keep inference functions independent of the UI so a local API can be added later.
- Pin library versions and document the Python version and installation commands.
- Measure inference latency and memory on the actual target machine before claiming real-time capability.
- Do not commit restricted datasets, private recordings, credentials or model artefacts that cannot be redistributed.

Suggested source tree

```
voice-emotion-recognition/
■■■ app/ (dashboard)
■■■ configs/ (dataset, features, model, training)
■■■ src/data/ (loading, labels, splits, audit)
■■■ src/audio/ (preprocessing, segmentation, features)
■■■ src/models/ (CNN, CNN-LSTM, optional alternatives)
■■■ src/inference/ (predict, uncertainty, timeline, suggestions)
■■■ src/evaluation/ (metrics, calibration, robustness, explainability)
■■■ tests/ (unit and integration tests)
■■■ artifacts/ (checkpoints, plots, reports)
■■■ requirements.txt
■■■ README.md
```

5. Dataset, preprocessing and experiment protocol

Candidate datasets

Dataset	Potential role	Checks before use
RAVDESS	Candidate primary acted-speech corpus	Confirm selected audio subset, labels, licence and terms
CREMA-D	Additional speakers and emotion examples	Check category mapping and recording properties
TESS	Supplementary examples	Check speaker/recording differences and overlap risks
SAVEE	Optional additional corpus	Inspect small speaker pool and class distribution
EMO-DB	Optional cross-corpus experiment	Account for language and recording-condition differences

Data protocol

- Start with one corpus and make the end-to-end pipeline work before merging datasets.
- Audit label counts, speaker IDs, sample rates, duration, channels, missing/corrupt files and licence terms.
- Map labels only when definitions are genuinely comparable; otherwise keep experiments separate.
- Split by speaker where possible; verify programmatically that speaker IDs do not overlap across train/validation/test.
- Use validation data for model selection and uncertainty thresholding; do not tune against the final test set.
- Record dataset version, preprocessing parameters, random seeds, class mapping, split IDs and model checkpoint.

Preprocessing choices to document

Step	Requirement
Sample rate / channels	Choose a target sample rate; convert to mono unless stereo is deliberately used
Duration	Keep original duration metadata; define minimum/maximum duration handling
Silence / VAD	Document whether silence is trimmed and which method is used
Segmentation	Choose window size and overlap through development experiments; preserve timestamps
Normalisation	Use consistent training/inference normalisation; avoid accidentally removing useful intensity cues
Denoising	Treat as an ablation because it may remove emotion-relevant acoustic details
Augmentation	Apply only to training data; document noise, pitch/time changes and limits

Evaluation metrics

Metric	Reason
Accuracy	Useful overall metric but can hide minority-class failures
Macro precision/recall/F1	Treat classes more equally; macro F1 is a useful primary metric under imbalance
Per-class recall	Shows which categories are missed
Confusion matrix	Shows patterns of class confusion
Calibration / reliability plot	Checks whether confidence aligns with observed correctness
Coverage vs selective risk	Measures the trade-off when uncertain cases are rejected
Robustness curves	Shows metric change as noise increases
Latency and memory	Checks practical deployment requirements

6. Experiment plan and acceptance criteria

Experiment	Question	Comparison / output
E1 — Baseline	Can the model learn dataset labels?	CNN–LSTM, speaker-independent test metrics and confusion matrix
E2 — Temporal model	Does LSTM help compared with CNN alone?	CNN vs CNN–LSTM on same split
E3 — Features	Do prosodic features add value?	Mel-only vs Mel + selected acoustic features
E4 — Timeline	Are segment-level outputs useful and stable?	Inspect timestamped examples; use segment metrics only if valid labels exist
E5 — Uncertainty	Can low-confidence errors be reduced?	Validation-selected thresholds; coverage-risk plot
E6 — Noise	How does performance degrade?	Clean vs multiple controlled noise conditions
E7 — Denoising	Does noise reduction help?	No denoising vs denoising
E8 — Speaker split	Does the model generalise to unseen speakers?	Speaker-independent results; optional comparison to random split clearly labelled
E9 — Explainability	Are visual explanations coherent and stable?	Qualitative case review; document method limitations
E10 — Personalisation	Does speaker baseline help?	Baseline on/off with consented reference data
E11 — Advanced model	Does attention or pretrained representation help?	Compare on identical splits and metrics if implemented
E12 — Cross-corpus	Does it transfer across corpora?	Only if common labels and licensing permit

Acceptance checklist (complete before final submission)

- Audio upload handles valid and invalid inputs without crashing.
- Feature shapes, label order and inference outputs are tested.
- Emotion timeline timestamps are ordered and fall within the recording duration.
- Uncertainty threshold is selected on validation data and tested on held-out data.
- Speaker-independent split is verified where speaker IDs are available.
- Every chart and score comes from saved experiment outputs; no illustrative value is presented as measured.
- Dashboard includes clear uncertainty and responsible-use notes.
- Robustness tests record the exact noise conditions used.
- Advanced modules are clearly labelled as implemented only after they work; unfinished modules remain future work.

Success is defined by a reproducible end-to-end system and evidence from fair experiments—not by a target accuracy number chosen in advance.

7. Implementation order and team roles

Phase	Work	Deliverable
1. Define scope	Confirm labels, primary dataset, permitted use, metrics and speaker split	Experiment specification
2. Audit data	Inspect labels, speakers, duration, file integrity and class balance	Dataset audit
3. Build audio pipeline	Implement preprocessing, segmentation and feature extraction	Repeatable audio pipeline
4. Train baselines	Train CNN and CNN–LSTM, save checkpoints and metrics	Baseline comparison
5. Build timeline	Run inference by segment and preserve timestamps	Working emotion timeline
6. Add uncertainty	Select validation threshold and create uncertain UI state	Coverage-risk evaluation
7. Robustness + explainability	Noise tests, feature plots, optional attribution	Robustness and explanation panels
8. Personalisation + suggestions	Consent-based speaker baseline; reviewed response templates	Advanced interaction modules
9. Dashboard integration	Connect upload, playback, predictions, timeline, evaluation and export	End-to-end application
10. Advanced comparisons	Attention/pretrained model; cross-corpus or multimodal only if feasible	Comparative experiments
11. Test and document	Run checks, write README, limitations, model card and test log	Reproducible release
12. Present	Demo clean/noisy/uncertain examples and discuss measured results	Final PPT + live demo

Suggested four-member split

Role	Primary ownership
Member 1 — Data pipeline	Dataset audit, label mapping, speaker split, preprocessing and permissions
Member 2 — Model	CNN/CNN–LSTM training, model comparisons and checkpoint management
Member 3 — Evaluation	Metrics, confusion matrix, uncertainty, robustness and experiment log
Member 4 — Application	Dashboard, timeline, visualisations, suggestions, report export and integration

All members should understand the end-to-end architecture and the results. The roles are ownership areas, not isolated silos.

8. Testing, ethics, limitations and presentation

Testing

Area	Test cases
Input	Empty/corrupt WAV, unsupported extension, long file, multiple sample rates, mono/stereo
Audio pipeline	Silence, short clips, finite feature values, stable feature dimensions
Model	Correct output shape, correct class mapping, repeatable inference
Timeline	Timestamp order, boundary handling, segment playback
Uncertainty	Scores above/below threshold, no-speech handling and threshold provenance
Data split	No speaker overlap; test data not used for tuning
Dashboard	Upload → play → predict → timeline → export works end to end
Deployment	Clean environment install, pinned requirements, latency measurement

Responsible use and privacy

- Obtain consent for any newly recorded voice and for any speaker reference used for personalisation.
- Process audio locally by default and avoid retaining raw audio unless needed and explicitly explained.
- Do not use the system for hiring, discipline, surveillance, diagnosis or other high-stakes decisions.
- Display a caveat that model predictions can be wrong and are not definitive statements of internal emotion.
- Report dataset limits, accent/language coverage, class imbalance and possible demographic performance gaps where evidence is available.
- Do not assume acted dataset emotions transfer to spontaneous conversation or across cultures.

Known limitations and mitigations

Limitation	Mitigation
Acted speech may not represent real interactions	State the limit; test other permitted data if possible
Speaker or recording artefacts may leak	Use speaker-independent split and cross-corpus tests when feasible
Confidence may be miscalibrated	Evaluate calibration and use a validation-based rejection rule
Denoising may remove useful cues	Run denoising ablation instead of assuming it helps
Labels differ across corpora	Use defensible common labels or separate experiments
Speaker baseline introduces privacy/bias risk	Consent, local processing, delete/reset, separate evaluation
Emotion suggestions may overinterpret audio	Use cautious templates and suppress suggestions for uncertain outputs

PPT update plan

Slide	Add or emphasise
Proposed solution	Segment-wise inference, uncertainty layer, timeline and dashboard
Methodology	Preprocessing → segmenting → features → CNN–LSTM → reliability → timeline
Innovation	Timeline, uncertainty handling, robustness, explainability and responsible-use design
Dataset	Label mapping, speaker-independent split and data terms
Evaluation	Macro F1, class metrics, confusion matrix, noise plots, coverage-risk; only measured values
Prototype	Real screenshots or a live demo; label mock-ups as illustrative
Limitations	Acted data, cross-speaker variation, uncertainty and privacy
Future work	Multilingual, multimodal, pretrained model and deployment extensions if not completed

9. Suggested tools and final feature checklist

Area	Suggested tools / approach
Audio I/O and features	librosa, soundfile, scipy; keep transformations consistent
Deep learning	TensorFlow/Keras or PyTorch—choose one dependable stack
Metrics	scikit-learn for classification metrics and confusion matrix
Plots	Matplotlib for reproducible figures
Dashboard	Streamlit for a quick local prototype; API/frontend can come later
Testing	pytest or repeatable test scripts
Experiment log	CSV/JSON with config, seed, split IDs, checkpoint, metrics and run notes
Version control	Git; exclude restricted audio, secrets and large private recordings

Full target feature checklist

- [ ] Audio upload and validation
- [ ] Preprocessing, segmentation and feature extraction
- [ ] CNN–LSTM model and CNN baseline
- [ ] Emotion timeline with timestamped predictions
- [ ] Uncertainty-aware predictions and validation-selected threshold
- [ ] Speaker-independent evaluation
- [ ] Noise robustness and denoising comparison
- [ ] Waveform, Mel-spectrogram and acoustic feature summaries
- [ ] Optional model-attribution heatmap, if implemented and validated
- [ ] Consent-based speaker reference and relative baseline
- [ ] Cautious, reviewable emotion-aware response suggestions
- [ ] Interactive dashboard and exportable summary
- [ ] Metrics, confusion matrix, calibration/coverage and latency reporting
- [ ] Reproducibility log, tests, README and model version information
- [ ] Privacy notice, limitations and responsible-use safeguards
- [ ] Advanced comparison: BiLSTM/attention or pretrained audio embeddings
- [ ] Advanced extension: cross-corpus/multilingual evaluation where data permit
- [ ] Advanced extension: multimodal fusion only with suitable consented data

Final recommendation

Treat this PDF as the complete target specification. Build in dependency order: first make the dataset, CNN–LSTM and evaluation reliable; then add the timeline, uncertainty layer and robustness experiments; then integrate explainability, speaker adaptation, response suggestions and dashboard; finally attempt advanced model comparisons and multimodal or multilingual extensions only if the core system is complete. Clearly distinguish implemented features from planned features in the final report and viva.

Basis: the supplied Deep Learning-Based Voice Emotion Recognition presentation and the improvement directions discussed in this conversation. This is a design blueprint; it does not claim that the listed features have already been implemented or evaluated.